Notes on Code

Ramzi Chariag

March 8, 2026

1 Guiding principles

1. **Mirror the economics, not the implementation details.** The code is organized by blocks (unskilled vs. skilled) and by computational layers (global link fixed point, block outer loops, inner value iterations), so that changes in the economics map cleanly into localized code edits.

2. **Immutable inputs; mutable workspace.** Parameters and grids are immutable (`@kwdef` structs with plain fields). Iterates and work arrays are mutable and live in `Cache` objects. This prevents accidental reallocation and makes performance predictable.

3. **Blocks do not "own" link variables.** The unskilled solver *consumes* $U_S(x)$ (read from `skl_cache.U`) and outputs $t(x)$, from which $m_S(x) = (\phi/\nu)\, t(x)$ is derived. The skilled solver *consumes* $m_S(x)$ and outputs an updated $U_S(x)$. This avoids circular ownership and keeps each block testable in isolation.

4. **Policies as cutoffs, not full boolean arrays.** We store cutoffs ($p_U^*(x)$, $p_S^*(x)$, $p_S^{oj}(x)$) and recover policies on the fly (e.g. $s^*(x,p) = \mathbb{1}\{p < p_S^{oj}(x)\}$). This reduces memory and simplifies updates.

5. **Separate economic updates from numerical stabilization.** Inner value-function loops use plain fixed-point iteration (no damping). The unskilled and skilled outer loops damp the cutoff updates (p_U^*, p_S^*, p^{oj}) via fixed mixing weights stored in `SimParams` (`damp_pstar_U`, `damp_pstar_S`). Anderson acceleration is applied only to the scalar tightness updates (both blocks) and to the global link-variable pair (U_S, m_S).

6. **Single regime.** The current implementation solves a single stationary equilibrium for one parameter regime. All objects are arrays over (x,p) without a regime index z. The regime-specific parameters live in `RegimeParams`.

2 Numerical policy: contraction vs. oscillatory layers

Inner value-function iterations: no damping by design. The value-function maps inside each block are treated as smooth contractions and iterated without damping. Concretely, the unskilled inner loop updates

$$U_U^{search}(x), \quad U_U(x), \quad T(x), \quad J_U^{frontier}(x)$$

by direct fixed-point iteration until the sup-norm change over all three is below `tol_inner` for `conv_streak` consecutive iterations.

The skilled inner loop updates

$$U_S(x), \quad E_S^0(x,p), \quad E_S^1(x,p), \quad J_S^0(x,p), \quad J_S^1(x,p)$$

by direct fixed-point iteration, with McQueen–Porteus extrapolation applied to U_S as an acceleration step (not damping—it speeds convergence by estimating the contraction ratio from successive increments).

Middle and top layers: oscillatory fixed points. The block outer loops and the global link loop can oscillate. The stabilization strategy is:

- **Cutoff updates (p_U^*, p_S^*, p_S^{oj}):** damped convex combinations controlled by `damp_pstar_U` (unskilled) and `damp_pstar_S` (skilled). Values outside $[0, 1]$ (over-relaxation / under-relaxation) are supported.
- **Tightness θ_U, θ_S:** Anderson acceleration with memory $m = 1$ (closed-form scalar update via `Anderson1`), toggled by `use_anderson`.
- **Global link loop (U_S, m_S):** Anderson acceleration with memory $m = 1$ applied to the concatenated $2N_x$-vector $[U_S;\, m_S]$, also toggled by `use_anderson`.

Convergence requires repeated success (streak rule). Each loop declares convergence only after the monitored sup-norm difference falls below tolerance for `conv_streak` consecutive iterations. This guards against transient improvements in oscillatory maps.

3 Three computational layers (nested fixed points)

Layer A: Global link fixed point (slowest). Link objects:

$$U_S(x) \quad \text{and} \quad m_S(x) = \frac{\phi}{\nu}\, t(x).$$

Map:

1. Input current U_S (stored in `skl_cache.U`) to the unskilled solver $\Rightarrow$ obtain $t(x)$ in `unsk_cache.t`, hence $m_S^{raw}(x) = (\phi/\nu)\, t(x)$.
2. Input m_S^{raw} to the skilled solver $\Rightarrow$ updated U_S^{raw} in `skl_cache.U`.
3. Apply Anderson$_1$ to the joint vector $[U_S;\, m_S]$:

   $$[U_S^{new};\, m_S^{new}] = \texttt{anderson1_update!}([U_S^{old};\, m_S^{old}],\ [U_S^{raw};\, m_S^{raw}]).$$

   Non-negativity of m_S^{new} is enforced. Convergence is measured by $\max\big(\|U_S^{new} - U_S^{old}\|_\infty,\ \|m_S^{new} - m_S^{old}\|_\infty\big)$.

Layer B: Block equilibrium solvers (medium). Unskilled outer loop. Objects updated each iteration:

$$\theta_U, \quad p_U^*(x), \quad u_U(x), \quad t(x), \quad \tau(x).$$

Steps:

1. *Inner loop*: run `unskilled_inner_loop!` to convergence, yielding $(U_U^{search}, U_U, T, J_U^{frontier})$ and the surplus integrals $I(x)$.
2. *Composition*: call `solve_stationary_unskilled_pointwise!` to update (u_U, t) from pointwise flow balance.

3. *Cutoff*: compute $p_U^{*,raw}(x)$ from the analytical formula, then damp: $p_U^* \leftarrow$ `damp_pstar_U`$\cdot p_U^{*,raw} + (1 - $`damp_pstar_U`$) \cdot p_U^{*,old}$.

4. *Tightness*: compute θ_U^{raw} from free entry; apply Anderson$_1$ on the scalar; update `unsk_cache.`θ.

Convergence is monitored jointly by $|\theta_U^{new} - \theta_U^{old}|$, $\|p_U^{*,new} - p_U^{*,old}\|_\infty$, and $\|u_U^{new} - u_U^{old}\|_\infty$.

Skilled outer loop. Objects updated each iteration:

$$\theta_S, \quad p_S^*(x), \quad p_S^{oj}(x), \quad u_S(x), \quad e_S(x,p).$$

Steps:

1. *Inner loop*: run `skilled_inner_loop!` to convergence, yielding $(U_S, E_S^0, E_S^1, J_S^0, J_S^1)$.

2. *Cutoffs*: compute $p_S^{*,raw}(x)$ (first p where $\max(S^0, S^1) \geq 0$) and $p_S^{oj,raw}(x)$ (first $p \geq p_S^*$ where $S^1(x,p) \leq S^0(x,p)$); then damp both with weight `damp_pstar_S`; enforce $p_S^{oj} \geq p_S^*$.

3. *Composition*: call `solve_stationary_skilled!` (threaded over x).

4. *Tightness*: compute θ_S^{raw} from free entry via `compute_Jbar_skilled`; apply Anderson$_1$ on the scalar; update `skl_cache.`θ.

Primary convergence criterion: $|\theta_S^{new} - \theta_S^{old}|$ (same convention as v1.0).

Layer C: Within-block inner solves (fastest).

- *Unskilled*: four sequential pointwise-in-x updates (T, U/τ, surplus integrals, U^{search}), threaded over x. The surplus integral $I(x)$ is solved analytically via a closed-form scalar fixed point (see Section 4).

- *Skilled*: tail-sum construction and pointwise ($p \times x$) updates for (U_S, E, J), threaded over x; McQueen–Porteus shift applied after each full sweep.

4 Unskilled match surplus: analytical reduction

In the unskilled block the match surplus satisfies

$$(r + \nu + \lambda)\, S_U(x,p) = P_U\, x\, p - (r + \nu)\, U_U(x) + \lambda\, I(x), \quad p \geq p_U^*(x),$$

where $I(x) = \int_{p_U^*(x)}^1 S_U(x,p)\, dG(p)$ and $G \sim \text{Beta}(\alpha_U, 1)$, so $G(p) = p^{\alpha_U}$ and $g(p) = \alpha_U p^{\alpha_U - 1}$.

Substituting the formula for S_U into the definition of I gives a scalar linear equation in I with coefficients

$$A = \frac{P_U\, x \int_{p^*}^1 p\, dG(p) - (r + \nu)\, U_U(x)\, \bar{G}(p^*)}{r + \nu + \lambda}, \qquad B = \frac{\lambda\, \bar{G}(p^*)}{r + \nu + \lambda},$$

yielding $I = A/(1-B)$ in closed form. This is solved analytically inside `solve_unskilled_surplus_on_grid!`; no inner iteration over p is required.

The reservation cutoff is then recovered pointwise:

$$p_U^*(x) = \frac{(r + \nu)\, U_U(x) - \lambda\, I(x)}{P_U\, x}, \quad \text{clamped to } [0,1].$$

The firm value at the frontier ($p = 1$) is $J_U^{frontier}(x) = (1 - \beta_U)\, S_U(x,1)$; free entry uses $\bar{J}_U = \int J_U^{frontier}(x)\, u_U(x)\, dx / \int u_U(x)\, dx$ to invert $q(\theta_U) = k_U/(\bar{J}_U/U_{total})$ (see `update_theta_unskilled`).

5 Skilled block details

Value update. Inside `skilled_inner_loop!`, for each x the tail sum

$$\mathrm{tailE}[j] = \sum_{k \geq j} \max\big(J_S^0[x,k], J_S^1[x,k]\big) \cdot w_\Gamma[k]/(1-\beta_S)$$

is built by a backward pass, giving $I(x) = \mathrm{tailE}[j_0(x)]$ where $j_0(x)$ is the first grid index above $p_S^*(x)$. Then:

$$U_S(x) = \frac{b_S + f_S\,\beta_S\,I(x)}{r+\nu},$$

$$S^0(x,p) = \frac{P_S\,x\,p - (r+\nu)\,U_S(x) + \lambda_S\,I(x)}{r+\nu+\xi_S+\lambda_S},$$

$$S^1(x,p) = \frac{P_S\,x\,p - (r+\nu)\,U_S(x) - \sigma_S + \lambda_S\,I(x) + f_S\,\beta_S\,\mathrm{tailE}[\max(j,j_0)]}{r+\nu+\xi_S+\lambda_S+f_S\,(1-\Gamma(p))},$$

and values are set by the Nash split: $E^k = U_S + \beta_S S^k$, $J^k = (1-\beta_S)S^k$.

McQueen–Porteus acceleration. After each full sweep over x, the span of the increments $\delta U_S(x) = U_S^{new}(x) - U_S^{old}(x)$ is computed. Denoting $\rho = \mathrm{span_new}/\mathrm{span_prev}$ (clamped below 1), a scalar shift $\rho/(1-\rho)\cdot(\delta_{min}+\delta_{max})/2$ is added to all $U_S(x)$. This is a standard Gauss–Seidel acceleration for contraction mappings.

On-the-job search cutoff. Workers search on the job if and only if $p < p_S^{oj}(x)$, where $p_S^{oj}(x)$ is the smallest $p \geq p_S^*(x)$ such that $S^1(x,p) \leq S^0(x,p)$. At low match quality the OJS option value exceeds its flow cost; at high quality the reverse holds.

Stationary distribution. `solve_stationary_skilled_x!` implements a forward sweep over the p-grid. For each $j \geq j_0(x)$, the mass e_j satisfies a linear relation $e_j = \alpha_j\,u_S(x) + \beta_j$ where (α_j, β_j) are computed recursively using cumulative sums of $\alpha_{j'}$, $\beta_{j'}$ for $j' < j$ (the "job-ladder" inflow from on-the-job search). After a full sweep, $u_S(x)$ is recovered from total-mass accounting:

$$u_S(x)\Big(1 + \sum_{j \geq j_0} \alpha_j\,w_p[j]\Big) = m_S(x) - \sum_{j \geq j_0} \beta_j\,w_p[j].$$

Free entry for skilled market. `compute_Jbar_skilled` computes

$$\bar{J}_S = \frac{\int \Big[u_S(x)\int_{p_S^*(x)}^1 J^*(x,p)\,d\Gamma(p) + \int_{p_S^*(x)}^1 s^*(x,p)\,e_S(x,p)\int_{\max(p,p_S^*(x))}^1 J^*(x,\tilde{p})\,d\Gamma(\tilde{p})\,dp\Big]\,dx}{\int\big[u_S(x) + \int s^*(x,p)\,e_S(x,p)\,dp\big]\,dx},$$

where $J^* = J^1$ if $E^1 \geq E^0$ (OJS job) and J^0 otherwise. If the numerator is near zero (early iterations when the stationary distribution is empty), a fallback uses the type density $\ell(x)$ as a proxy for seeker mass.

6 Concrete list of structs

6.1 Parameter and grid structs (immutable)

Common.

- `CommonParams`: r, ν, ϕ; Beta shape parameters (a_ℓ, b_ℓ) for the worker-type density $\ell(x) \sim \text{Beta}(a_\ell, b_\ell)$; training cost coefficient c (so that $c(x) = c\,e^{-6x}$, decreasing in x).
- `RegimeParams`: productivity shifters P_U, P_S; flow payoffs b_U, b_T, b_S; unskilled damage-shock Beta shape α_U (so $G \sim \text{Beta}(\alpha_U, 1)$, i.e. $G(p) = p^{\alpha_U}$); skilled offer-quality Beta shapes a_Γ, b_Γ.
- `CommonGrids`: Gauss–Legendre nodes x_i on $[0, 1]$, weights w_i^x, and $\ell(x_i)$.
- `UnskilledParams`: $\mu_U, \eta_U, k_U, \beta_U, \lambda_U$.
- `UnskilledGrids`: Gauss–Legendre nodes p_j^U and weights w_j^U.
- `SkilledParams`: $\mu_S, \eta_S, k_S, \beta_S, \xi_S, \lambda_S, \sigma_S$.
- `SkilledGrids`: Gauss–Legendre nodes p_j^S and weights w_j^S.

Precomputed (skilled only).

- `SkilledPrecomp`: $\Gamma(p_j)$, $\gamma(p_j)$, and tail weights tail_weights$[j] = \sum_{k \geq j} w_k^S\, \gamma(p_k)$ (tail mass of $d\Gamma$). These are computed once in `build_skilled_precomp`.

Simulation controls.

- `SimParams`: all tolerances (`tol_inner`, `tol_outer_U`, `tol_outer_S`, `tol_global`); iteration caps (`maxit_inner`, `maxit_outer`, `maxit_global`); `conv_streak`; Anderson toggle `use_anderson` and memory `anderson_m` (currently always 1); `anderson_reg`; damping weights `damp_pstar_U` and `damp_pstar_S`; verbosity controls `verbose` and `verbose_stride`.

6.2 Cache structs (mutable)

UnskilledCache.

- Values: $U_U^{search}(x)$, $U_U(x)$, $T(x)$.
- Match: $J_U^{frontier}(x)$ (firm value at $p = 1$).
- Policies: $p_U^*(x)$, $\tau(x)$.
- Composition: $u_U(x)$, $t(x)$.
- Scalar: θ_U.

Note: the full surplus surface $S_U(x, p)$ and employment surface $e_U(x, p)$ are *not* stored in the cache; they are recomputed on demand (see `compute_equilibrium_objects`).

SkilledCache.

- Values: $U_S(x)$; $E_S^0(x, p)$, $E_S^1(x, p)$ (worker values without/with OJS).
- Firm values: $J_S^0(x, p)$, $J_S^1(x, p)$.
- Policies: $p_S^*(x)$, $p_S^{oj}(x)$.
- Composition: $u_S(x)$, $e_S(x, p)$.
- Scalar: θ_S.

Note: wages are *not* stored in the cache. They are recovered from the Nash formula in `compute_equilibrium_objects` after the solver has converged.

6.3 Top-level model container

- `Model`: holds `common`, `regime`, `grids`, `unsk_par`, `unsk_grids`, `unsk_cache`, `skl_par`, `skl_grids`, `skl_pre`, `skl_cache`, `sim`.

7 Functions (solver API)

Construction and initialization.

- `build_gl_grid(N)`: Gauss–Legendre nodes and weights on $[0, 1]$ via `FastGaussQuadrature.gausslegendre`.
- `build_skilled_precomp(sg, rp)`: fills `SkilledPrecomp` from grid and `RegimeParams`.
- `_initialise_caches(...)`: allocates and seeds `UnskilledCache` and `SkilledCache`. Unskilled initial conditions: $U_U^{search} = b_U/(r+\nu)$, $T = (b_T + \phi\, U_S^{guess})/(r+\phi+\nu)$ where $U_S^{guess} = b_S/(r+\nu)$; $U_U = \max(U_U^{search}, T)$; $p_U^* = 0.1$; $\theta_U = 1$; $u_U = 0.4\,\ell$. Skilled: $U_S = b_S/(r+\nu)$; $p_S^* = 0.1$; $p_S^{oj} = 0.6$; $\theta_S = 1$.
- `initialise_model(; Nx, Np_U, Np_S, regime)`: public constructor, returns a ready-to-solve `Model`.

Top-level solve.

- `solve_model(common, regime, unsk_par, skl_par, sim; Nx, Np_U, Np_S)`: builds grids, caches, and runs the global loop; returns the solved `Model`.
- `solve_model!(model)`: in-place variant for an already-initialized `Model`. Both functions implement the same global Algorithm A above.

Block solvers.

- `solve_unskilled_block!(model; US_in)`: unskilled outer loop consuming $U_S =$ `US_in`, updating `unsk_cache` in-place.
- `solve_skilled_block!(model; mS_in)`: skilled outer loop consuming $m_S =$ `mS_in`, updating `skl_cache` in-place.

Inner-loop kernels.

- `unskilled_inner_loop!(model; US_in)`: four-step fixed point over $(U_U^{search}, U_U, T, J_U^{frontier})$; returns `(f, Ivec)`.
- `solve_unskilled_surplus_on_grid!(Svec, pgrid, wG, PU, x, r, ν, λ, Ux, pstar_x)`: analytical surplus solve for one x; returns `(I, tail_mass, i0)`.
- `skilled_inner_loop!(model; mS_in)`: sweep over x updating $(U_S, E^0, E^1, J^0, J^1)$ with McQueen–Porteus; returns `(f,)`.
- `solve_stationary_unskilled_pointwise!(u_out, t_out, ...)`: pointwise flow balance for (u_U, t).
- `update_pstar_from_surplus!(pstar_new, model, Ivec)`: analytical formula for $p_U^*(x)$.
- `update_theta_unskilled(model)`: free-entry inversion for θ_U.

- `solve_stationary_skilled_x!(ix, model, mS_x)`: forward sweep for $e_S(x,\cdot)$ and $u_S(x)$ at a single x.
- `solve_stationary_skilled!(model; mS_in)`: threaded wrapper calling `solve_stationary_skilled_x!` for all x.
- `compute_Jbar_skilled(model)`: computes $\bar{J}_S$ for free entry.
- `update_theta_skilled(model)`: free-entry inversion for θ_S.

Utility functions.

- `q_from_theta(`θ`, `μ`, `η`)` $= \mu\,\theta^{-\eta}$, `theta_from_q(`q`, `μ`, `η`)` $= (\mu/q)^{1/\eta}$, `jobfinding_rate(`θ`, `μ`, `η`)` $= \theta\, q(\theta)$.
- `build_gl_grid(N)`: quadrature nodes/weights.
- `build_unskilled_G_weights(pgrid, wp, `α`)`: weights $w_j\, g(p_j)$ for $G \sim \mathrm{Beta}(\alpha, 1)$.
- `G_cdf_unskilled(`p`, `α`)` $= p^{\alpha}$.
- `build_tail_weights(`$w\Gamma$`)`: reverse prefix sum giving $\mathrm{tail}[j] = \sum_{k\geq j} w\Gamma[k]$.
- `pcut_index(pgrid, pstar)`: first grid index j with $p_j \geq pstar$.
- `find_cutoff_from_j0(pgrid, Smax, j0_prev)`: warm-started forward search for p_S^* (first p where $\max(S^0, S^1) \geq 0$).
- `find_poj_from_diff_grid(pgrid, diff, pstar)`: first $p \geq pstar$ where $S^1 - S^0 \leq 0$ (OJS cutoff).
- `training_cost(`x`, `c`)` $= c\, e^{-6x}$.
- `supnorm(`a`, `b`)` $= \|a - b\|_\infty$; `clamp01(`x`)`.
- `Anderson1(n)`, `anderson1_update!(aa, x, f)`: memory-1 Anderson in closed form.

Post-processing.

- `compute_equilibrium_objects(model)`: after the solver converges, computes the full set of equilibrium objects—value and surplus surfaces, employment distributions, wages $w_U(x,p)$, $w_S^0(x,p)$, $w_S^1(x,p)$, wage densities by sector, and population accounting aggregates—returned as a named tuple.

8 Wages (recovered post-solution)

Wages are not stored during the solve; they are computed in `compute_equilibrium_objects` from the Nash formula evaluated on the solved values:

$$w_U(x,p) = \beta_U\, P_U\, x\, p + (1-\beta_U)(r+\nu)\, U_U(x), \quad p \geq p_U^*(x),$$

$$w_S^0(x,p) = \beta_S\, P_S\, x\, p + (1-\beta_S)\, b_S + \beta_S(1-\beta_S)\, \kappa_S\, I(x), \quad p \geq p_S^*(x),$$

$$w_S^1(x,p) = \beta_S\, P_S\, x\, p + (1-\beta_S)(b_S + \sigma_S) + \beta_S(1-\beta_S)\, \kappa_S \left[I(x) - \mathrm{tailS}(x,p)\right], \quad p \geq p_S^*(x),$$

where $\kappa_S = \theta_S\, q(\theta_S)$ is the job-finding rate, $I(x)$ is the tail integral of the optimal surplus above $p_S^*(x)$, and $\mathrm{tailS}(x,p)$ is the same tail integral starting from p (OJS option value already realized above p). The wage differential $\Delta w(x,p) = w_S^1(x,p) - w_S^0(x,p)$ is negative: OJS workers accept lower base wages in exchange for the option to trade up.

9 Cell layout of the notebook

- **Cell 1** (Imports): packages (`LinearAlgebra`, `Distributions`, `FastGaussQuadrature`, `Interpolations`, `NLsolve`, etc.).
- **Cell 2** (Structs): all `@kwdef` struct definitions.
- **Cell 3** (Utilities): matching functions, norms, `build_gl_grid`, `training_cost`, `build_tail_weights`, `pcut_index`, `Anderson1`.
- **Cell 4** (Unskilled inner loop): `build_unskilled_G_weights`, `G_cdf_unskilled`, `solve_unskilled_surplus_on_grid!`, `unskilled_inner_loop!`.
- **Cell 5** (Unskilled outer loop): `solve_stationary_unskilled_pointwise!`, `update_pstar_from_surplus!`, `update_theta_unskilled`, `solve_unskilled_block!`.
- **Cell 6** (Skilled precompute and cutoffs): `build_skilled_precomp`, `find_cutoff_from_j0`, `find_poj_from_diff_grid`.
- **Cell 7** (Skilled inner and outer loops): `skilled_inner_loop!`, `solve_stationary_skilled_x!`, `solve_stationary_skilled!`, `compute_Jbar_skilled`, `update_theta_skilled`, `solve_skilled_block!`.
- **Cell 8** (Global solver): `_initialise_caches`, `solve_model`, `solve_model!`.
- **Cell 9** (Parameterize and solve): `initialise_model`, calibration values, call to `solve_model!`.
- **Cell 10** (Equilibrium objects): `compute_equilibrium_objects`, wage recovery, employment surfaces, population accounting.

10 Default calibration values

For reference, the baseline parameter values set in `initialise_model`:

Common: $r = 0.05$, $\nu = 0.05$, $\phi = 0.20$; $\ell \sim \text{Beta}(2, 5)$; $c = 1.70$.

Regime: $P_U = 0.70$, $P_S = 1.85$; $b_U = 0.00$, $b_T = 0.28$, $b_S = 0.01$; $\alpha_U = 1.00$ ($G \sim \text{Uniform}[0, 1]$); $\Gamma \sim \text{Beta}(2, 5)$.

Unskilled: $\mu_U = 0.74$, $\eta_U = 0.60$, $k_U = 0.25$, $\beta_U = 0.40$, $\lambda_U = 0.08$.

Skilled: $\mu_S = 0.90$, $\eta_S = 0.50$, $k_S = 0.17$, $\beta_S = 0.32$, $\xi_S = 0.03$, $\lambda_S = 0.07$, $\sigma_S = 0.01$.

Simulation: `tol_inner`$= 10^{-8}$, `tol_outer_U`$= 10^{-6}$, `tol_outer_S`$= 10^{-5}$, `tol_global`$= 10^{-3}$; `maxit_inner`$= 500$, `maxit_outer`$= 300$, `maxit_global`$= 50$; `conv_streak`$= 2$; `use_anderson`$=$ `true`, `anderson_m`$= 1$; `damp_pstar_U`$= 1.30$ (over-relaxed), `damp_pstar_S`$= 0.02$ (very tight); grid sizes $N_x = N_p^U = N_p^S = 200$.

11 Implementation notes: performance hygiene

- All large arrays are allocated once in caches and reused across iterations.
- Thread-local buffers (e.g. `Svec_tls`, `tailE_tls`) are pre-allocated outside inner loops to avoid per-iteration allocations.
- Threading is over the x-dimension (`Threads.@threads for ix in 1:Nx`), both in value updates and in the stationary distribution solve.

- The p-grid loops are sequential (backward or forward passes with dependencies).
- Tail sums are built by a single backward pass per x-type ($O(N_p)$ per x), not by repeated integration.
- Outer fixed-point loops are sequential; all parallelism is inside per-x work.
- `@inbounds` is used throughout inner loops; bounds checks are left on for outer-level diagnostic code.